



UNIVERSIDAD DE LOS LAGOS

LABORATORIO PRÁCTICO N°2

IMPLEMENTACIÓN DE STACK WEB CON DOCKER

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
TALLER DE INGENIERÍA INFORMÁTICA
CAMPUS OSORNO, CHILE

Felipe Alejandro Pino Díaz
Vicente Ignacio Salazar González
Benjamín Arturo Poblete Ibarra

1 de junio de 2026



5 años
UNIVERSIDAD ACREDITADA
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1. Introducción	IV
2. Desarrollo	V
2.1. Arquitectura del Stack Web	V
2.2. Aislamiento de Redes y Persistencia de Datos	V
2.3. Evidencia de Desarrollo y Despliegue	VI
2.4. Demostración de Funcionalidad CRUD	VII
3. Conclusiones	IX
4. Anexos	X

Índice de figuras

2.1. Ejecución del comando de construcción de imágenes o levantamiento de servicios (<code>docker-compose build -d</code>).	VI
2.2. Ejecución del comando <code>docker-compose up -d</code> y verificación de contenedores activos mediante el comando <code>docker ps</code>	VII
2.3. Ejecución del comando <code>docker compose logs</code> para verificar que el servicio MySQL inició correctamente.	VII
2.4. Captura de la aplicación corriendo en el navegador	VIII

Índice de algoritmos

4.1. Archivo Dockerfile de la Aplicación	X
4.2. Archivo de orquestación docker-compose.yml	X

Capítulo 1

Introducción

En el paradigma actual del desarrollo de software, la contenerización se ha establecido como un estándar industrial para garantizar la portabilidad, escalabilidad y consistencia de las aplicaciones en múltiples entornos. El presente informe documenta el desarrollo y despliegue del Laboratorio Práctico N°2, cuyo objetivo central es la implementación de un stack web dinámico utilizando herramientas de orquestación con Docker. Para cumplir con este propósito, se ha diseñado una arquitectura compuesta por un contenedor para el servidor web (PHP) y otro para el motor de base de datos relacional (MySQL), interconectados de forma segura mediante una red virtual privada y utilizando volúmenes para asegurar la persistencia de la información. A nivel aplicativo, el proyecto se materializa en un sistema web de gestión para un Refugio de Mascotas". Esta plataforma funcional permite ejecutar de manera integral las cuatro operaciones fundamentales de acceso a datos (CRUD: Crear, Leer, Actualizar y Borrar), facilitando la administración eficiente de los registros de los animales, sus características y la actualización de su estado de adopción en tiempo real.

Capítulo 2

Desarrollo

2.1. Arquitectura del Stack Web

En esta arquitectura, el contenedor de la aplicación web (PHP) y el motor de base de datos relacional (MySQL) operan de manera independiente, pero se comunican de forma segura a través de una red virtual interna (network) orquestada por Docker. Al estar declarados dentro de la misma red en el archivo `docker-compose.yml`, Docker provee un servicio de resolución DNS interno y automático.

Esto significa que el código PHP no necesita conocer la dirección IP de la base de datos para establecer la conexión; simplemente utiliza el nombre del servicio del contenedor MySQL (por ejemplo, `db` o `mysql`) como nombre de host (`hostname`). Esta topología de red aislada garantiza la seguridad del sistema, ya que el contenedor de la base de datos no expone sus puertos al exterior, recibiendo únicamente las peticiones internas que provienen del contenedor de la aplicación web.

2.2. Aislamiento de Redes y Persistencia de Datos

Para garantizar la seguridad y eficiencia de la infraestructura, el proyecto implementa un estricto aislamiento mediante las capacidades de `networks` en Docker Compose. Se definió una red virtual interna (de tipo *bridge*) a la cual ambos contenedores —la aplicación web y el motor de base de datos— están vinculados. Esta configuración aísla a MySQL del exterior, ya que no requiere exponer sus puertos hacia la red pública de la máquina anfitriona, sino únicamente hacia esta subred privada. Además, esta red proporciona un servicio de resolución de nombres (DNS) automático, permitiendo que el código PHP se conecte a la base de datos referenciando directamente el nombre del servicio definido en el archivo de orquestación, prescindiendo del uso de direcciones IP estáticas y mejorando la portabilidad del entorno.

Por otro lado, para resolver la naturaleza efímera de los contenedores, se implementó una estrategia de almacenamiento a través de `volumes`. Se configuró un volumen que vincula un espacio seguro y persistente en el disco de la máquina anfitriona (Debian) con la ruta interna `/var/lib/mysql` dentro del contenedor de la base de datos. Esta directiva es fundamental, ya que asegura que toda la información crítica gestionada por el sistema “Re-

fugio de Mascotas” —tales como el ingreso de nuevos animales, la edición de sus perfiles y la actualización de sus estados de adopción— se almacene de forma permanente. De este modo, se garantiza que ante cualquier detención, reinicio o destrucción del contenedor de MySQL, la integridad de la base de datos se mantenga intacta y completamente disponible para futuros despliegues.

2.3. Evidencia de Desarrollo y Despliegue

A continuación se presentan las capturas de pantalla de la consola Linux (obtenidas a través de Bitwise SSH Client) que validan el ciclo de vida y estado del despliegue.

A continuación, se detalla el proceso de construcción de la imagen personalizada para el contenedor de la aplicación web. Al ejecutar el comando `docker compose build --no-cache`, se fuerza la compilación desde cero omitiendo el uso de caché. Como se observa en la consola, este proceso descarga la imagen base de PHP con Apache y ejecuta las directivas del Dockerfile, instalando con éxito las extensiones fundamentales de PHP (como `pdo` y `pdo_mysql`) necesarias para la conexión a la base de datos, además de habilitar el módulo `rewrite` de Apache.

```
root@debian:~/lab-crud# docker compose build --no-cache
[+] Building 22.2s (9/9) FINISHED
=> [internal] load local bake definitions           0.0s
=> reading from stdin 592B                         0.0s
=> [internal] load build definition from Dockerfile  0.0s
=> transferring dockerfile: 252B                    0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 1.6s
=> [internal] load .dockerignore                    0.0s
=> transferring context: 2B                          0.0s
=> CACHED [1/3] FROM docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd 0.1s
=> resolve docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd 0.0s
=> [2/3] RUN docker-php-ext-install pdo pdo_mysql 18.6s
=> [3/3] RUN a2enmod rewrite                        0.5s
=> exporting to image                               0.8s
=> exporting layers                                 0.4s
=> exporting manifest sha256:258f1920f8cef08439659857216fcc9d3a28416f270b1d40265eb9bd9343906 0.1s
=> exporting config sha256:90c8578c3237dca79fcfb03b81a896647b119c44ebcb0cf11aefd16b38712949 0.0s
=> exporting attestation manifest sha256:083736486fd846259e4681e2102e028e07987268b6c89e4a73535c02ba20f266 0.1s
=> exporting manifest list sha256:895705d32672c294e463d7bd899cd942f83b1ebbf761f4d1a6b961c8e74ab78c 0.0s
=> naming to docker.io/library/lab-crud-app:latest 0.0s
=> unpacking to docker.io/library/lab-crud-app:latest 0.1s
=> resolving provenance for metadata file           0.0s
[+] build 1/1
Image lab-crud-app Built
root@debian:~/lab-crud#
```

Figura 2.1: Ejecución del comando de construcción de imágenes o levantamiento de servicios (`docker-compose build -d`).

Una vez finalizada la etapa de construcción, se procede a levantar la infraestructura completa mediante el comando `docker compose up -d`. Esta instrucción crea la red virtual interna `lab-crud_crud-net` e inicializa los contenedores en segundo plano. Inmediatamente después, la ejecución de `docker compose ps` permite comprobar que tanto el contenedor de la aplicación (`lab-crud-app`) como el de la base de datos (`lab-crud-db`) se encuentran activos y en ejecución (estado `Up`). Asimismo, se verifica el correcto mapeo de puertos, confirmando que el puerto 8000 de la máquina anfitriona redirige al puerto 80 del contenedor web.

```

root@debian:~/lab-crud# docker compose up -d
[+] up 3/3
  ✓ Network lab-crud_crud-net Created                                0.1s
  ✓ Container lab-crud-db Started                                    0.5s
  ✓ Container lab-crud-app Started                                    1.0s
root@debian:~/lab-crud# docker compose ps
NAME                IMAGE             COMMAND                  SERVICE    CREATED         STATUS         PORTS
lab-crud-app        lab-crud-app      "docker-php-entrypoi..." app        About a minute ago Up About a minute 0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
lab-crud-db         mysql:8.0         "docker-entrypoint.s..." db         About a minute ago Up About a minute 3306/tcp, 33060/tcp

```

Figura 2.2: Ejecución del comando `docker-compose up -d` y verificación de contenedores activos mediante el comando `docker ps`.

Finalmente, para certificar que el motor de almacenamiento funciona de manera óptima dentro de su contenedor, se inspeccionan sus registros internos utilizando el comando `docker compose logs db | tail -30`. La salida desplegada en la consola demuestra que el servicio de MySQL versión 8.0 ha completado su fase de inicialización de manera íntegra, mostrando el mensaje de confirmación *ready for connections* en el puerto 3306. Esto garantiza que el backend de la base de datos está completamente operativo y dispuesto para procesar las operaciones CRUD.

```

root@debian:~/lab-crud# docker compose logs db | tail -30
lab-crud-db | 2026-06-01 00:02:55+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab-crud-db | 2026-06-01 00:02:56+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
lab-crud-db | 2026-06-01 00:02:56+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab-crud-db | ' /var/lib/mysql/mysql.sock' -> '/var/run/mysql/mysql.sock'
lab-crud-db | 2026-06-01T00:02:57.131665Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
lab-crud-db | 2026-06-01T00:02:57.132605Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting as process 1
lab-crud-db | 2026-06-01T00:02:57.143062Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
lab-crud-db | 2026-06-01T00:02:58.321356Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
lab-crud-db | 2026-06-01T00:02:58.956813Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
lab-crud-db | 2026-06-01T00:02:58.956980Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
lab-crud-db | 2026-06-01T00:02:58.965213Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysql' in the path is accessible to all OS users. Consider choosing a different directory.
lab-crud-db | 2026-06-01T00:02:59.014659Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysql/mysql.sock
lab-crud-db | 2026-06-01T00:02:59.015268Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46' socket: '/var/run/mysql/mysql.sock' port: 3306 MySQL Community Server - GPL.
root@debian:~/lab-crud#

```

Figura 2.3: Ejecución del comando `docker compose logs` para verificar que el servicio MySQL inició correctamente.

2.4. Demostración de Funcionalidad CRUD

En esta sección se documenta el correcto funcionamiento de la interfaz web de la aplicación dinámica conectada al motor de base de datos relacional.

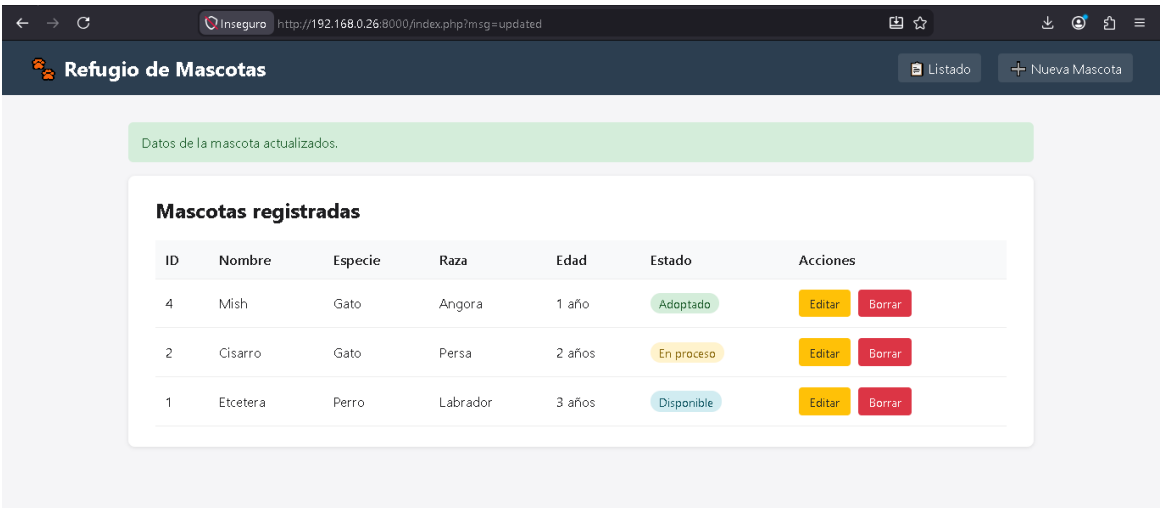


Figura 2.4: Captura de la aplicación corriendo en el navegador

Capítulo 3

Conclusiones

La realización de este laboratorio práctico permitió consolidar con éxito los conceptos fundamentales de orquestación de contenedores y diseño de infraestructura web aislada. Mediante el uso de Docker Compose, se logró desplegar un entorno de desarrollo robusto y reproducible, superando el clásico problema de incompatibilidad de entornos locales. La correcta implementación de volúmenes garantizó que la persistencia de los datos sobreviviera al ciclo de vida de los contenedores, mientras que la configuración de la red interna de Docker demostró ser una solución eficaz para la resolución de nombres (DNS) y la comunicación segura entre el backend en PHP y el servidor MySQL. Finalmente, el despliegue de la aplicación "Refugio de Mascotas" validó el cumplimiento técnico y funcional del proyecto, evidenciando un dominio práctico sobre el ciclo completo de operaciones CRUD a través de una interfaz web dinámica. En conclusión, la integración de estas tecnologías de contenerización establece una base metodológica sólida, escalable y profesional, indispensable para el desarrollo de futuros proyectos de ingeniería de software.

Capítulo 4

Anexos

A continuación, se detalla el archivo `Dockerfile` utilizado para la construcción de la imagen personalizada del contenedor de la aplicación:

Código 4.1: Archivo Dockerfile de la Aplicación

```
1 FROM php:8.2-apache
2
3 # Instalar la extensión PDO para conectarse a MySQL
4 RUN docker-php-ext-install pdo pdo_mysql
5
6 # Habilitar mod_rewrite (útil si después queremos URLs limpias)
7 RUN a2enmod rewrite
8
9 EXPOSE 80
```

Del mismo modo, el archivo `docker-compose.yml` define y orquesta ambos servicios (App y BD), asegurando las variables de entorno, la persistencia y la red aislada:

Código 4.2: Archivo de orquestación docker-compose.yml

```
1 services:
2   app:
3     build: ./app
4     container_name: lab-crud-app
5     ports:
6       - "8000:80"
7     depends_on:
8       - db
9     environment:
10       DB_HOST: db
11       DB_NAME: ${MYSQL_DATABASE}
12       DB_USER: ${MYSQL_USER}
13       DB_PASSWORD: ${MYSQL_PASSWORD}
14     volumes:
15       - ./app:/var/www/html
```

```
16     networks:
17         - crud-net
18     restart: unless-stopped
19
20 db:
21     image: mysql:8.0
22     container_name: lab-crud-db
23     environment:
24         MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
25         MYSQL_DATABASE: ${MYSQL_DATABASE}
26         MYSQL_USER: ${MYSQL_USER}
27         MYSQL_PASSWORD: ${MYSQL_PASSWORD}
28     volumes:
29         - db-data:/var/lib/mysql
30         - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
31     networks:
32         - crud-net
33     restart: unless-stopped
34
35 volumes:
36     db-data:
37
38 networks:
39     crud-net:
40         driver: bridge
```

Para la revisión completa del código fuente y del documento de diseño, se ponen a disposición los siguientes accesos funcionales obligatorios:

- **Enlace al Repositorio GitHub:** <https://github.com/Poblete999/lab2-docker-web.git>
- **Enlace al Proyecto LaTeX en Overleaf (Modo Lectura):** <https://www.overleaf.com/read/zqwsxqwfqkfz#614480>